GNR 638: Machine Learning for Remote Sensing - II

Programming Assignment 1: Design a Deep Learning Framework

Aman Nehra (23B1051)
Abhinav Singh (23B1003)
Yash Singh (23B0934)

February 15, 2026

Contents

1 Introduction **3**

2 Framework Design and Implementation **3**
2.1 C++ Backend Architecture 3
2.1.1 Tensor Abstraction 3
2.1.2 Parallel Data Loading 4
2.2 Python Frontend and Integration 4
2.3 Implemented Layers and Operations 4
2.4 Optimization 5

3 Dataset Loading and Processing **5**
3.1 Custom DataLoader Implementation 5
3.2 Preprocessing Pipeline 6
3.3 Loading Time Analysis 6

4 Model Architecture and Complexity **6**
4.1 Architecture Design 6
4.2 Design Rationale 7
4.3 Complexity Analysis 7
4.3.1 Formulas Used 7
4.3.2 Reported Metrics 8

5 Experimental Results and Evaluation **8**
5.1 Training Performance: Dataset 1 9
5.2 Training Performance: Dataset 2 9
5.3 Evaluation on Dataset 1 10
5.4 Evaluation on Dataset 2 10

6 Discussion and Analysis **11**
6.1 Failed Design Decision: Architectural Complexity vs. Training Constraints 11
6.2 Successful Optimizations 12

7 Conclusion **12**

1 Introduction

This programming assignment focuses on the fundamental principles of deep learning by requiring the design and implementation of a custom deep learning framework from scratch [1]. The primary objective is to build a framework capable of defining, training, and evaluating a Convolutional Neural Network (CNN) for multi-class image classification without relying on existing deep learning libraries such as PyTorch, TensorFlow, or Keras [1].

The assignment strictly prohibits the use of automatic differentiation engines or high-level neural network APIs [1]. Instead, all core components—including tensor operations, forward propagation, backpropagation, and optimization algorithms—must be implemented from first principles. To ensure computational efficiency and meet the strict training time constraints (less than 3 hours per dataset), the framework utilizes a hybrid architecture: a high-performance C++ backend for computationally intensive operations and a Python frontend for ease of experiment configuration and data management [1].

By completing this assignment, several key learning outcomes were achieved [1]:

- **Deep Learning Fundamentals:** Gained a granular understanding of the mathematical operations underlying neural networks, particularly the mechanics of backpropagation and gradient descent [2].
- **System Design:** Developed a modular framework that integrates a C++ computational engine with a Python interface using standard bindings [4].
- **Performance Optimization:** Implemented parallel data loading and vectorized operations to handle large datasets efficiently.
- **Model Analysis:** Analyzed model complexity in terms of learnable parameters, Multiply-Accumulate operations (MACs), and Floating Point Operations (FLOPs).

2 Framework Design and Implementation

The custom framework, named `custom_core`, follows a layered architecture designed to balance performance with usability.

2.1 C++ Backend Architecture

The core computational engine is implemented in C++ to leverage its speed and memory management capabilities, as strongly encouraged by the assignment guidelines for performance reasons [1].

2.1.1 Tensor Abstraction

The fundamental data structure is the `Tensor` class. It is implemented as a flattened 1D `std::vector<float>` to represent N-dimensional tensors (e.g., $N \times C \times H \times W$ for images). This design ensures contiguous memory allocation, which is cache-friendly and improves performance during iteration.

- **Data Storage:** `vec_data` stores the forward pass activations.

- **Gradient Storage:** `vec_grad` accumulates gradients during the backward pass [1].
- **Momentum Storage:** `vec_vel` stores velocity vectors for the SGD with Momentum optimizer [2].

The class provides essential methods such as `zero_grad()` to reset gradients before each training step and `grad_abs_sum()` for debugging gradient flow.

2.1.2 Parallel Data Loading

To prevent I/O bottlenecks, the data loading pipeline uses C++ multithreading via `std::async` and `std::future`. The dataset is divided into chunks, and multiple threads read and process images concurrently using OpenCV, which is exclusively permitted for image loading [1, 5].

- **Image Reading:** Images are read from disk using `cv::imread`.
- **Preprocessing:** Images are resized to 32×32 pixels and normalized to the range $[-1, 1]$ using the formula: $(pixel/255.0 - 0.5) \times 2.0$.
- **Format:** The loaded data is converted into $1 \times 3 \times 32 \times 32$ tensors (NCHW format) ready for processing [1].

2.2 Python Frontend and Integration

The Python frontend provides a high-level API for defining models and running training loops. The integration is achieved using `pybind11`, which exposes the C++ classes and functions as a Python module [4]. This allows the user to write training scripts in Python (e.g., `train_dataset_1.py`) while the heavy lifting happens in compiled C++.

2.3 Implemented Layers and Operations

Since automatic differentiation libraries are prohibited [1], the backward pass for every operation was derived and implemented manually.

1. **Convolution (Conv2d):**
   - *Forward:* Implements a sliding window operation with support for stride and padding.
   - *Backward:* Computes gradients with respect to the input (for the previous layer), weights, and biases [2].
2. **Fully Connected (Linear):**
   - *Forward:* Performs matrix multiplication $Y = XW^T + b$.
   - *Backward:* Propagates gradients to inputs ($dL/dX = dL/dY \cdot W$) and computes weight updates ($dL/dW = (dL/dY)^T \cdot X$) [2].
3. **Max Pooling:**

- *Forward:* Downsamples the input by selecting the maximum value in each window [2]. It also stores a binary `mask` indicating the position of the maximum value.
- *Backward:* Uses the stored `mask` to route gradients only to the input pixels that contributed to the maximum value (Sparse Gradient Propagation).

4. **Activation (ReLU):**
   - *Forward:* Applies $f(x) = \max(0, x)$ element-wise [2].
   - *Backward:* Zeroes out gradients where the input was ≤ 0.
5. **Loss Function (Softmax Cross-Entropy):**
   - Combined Softmax and Cross-Entropy for numerical stability [2].
   - *Gradient:* $\frac{\partial L}{\partial z_i} = p_i - y_i$, where p_i is the predicted probability and y_i is the one-hot encoded label.

2.4 Optimization

The framework implements **Stochastic Gradient Descent (SGD) with Momentum**. Momentum helps accelerate gradient vectors in the right direction, thus leading to faster convergence [2].

$$v_{t+1} = \mu \cdot v_t + g_{t+1} \tag{1}$$

$$w_{t+1} = w_t - \eta \cdot v_{t+1} \tag{2}$$

Where μ is the momentum factor (set to 0.9), η is the learning rate, and g is the gradient (including L2 regularization/weight decay).

3 Dataset Loading and Processing

Efficient data loading is critical for meeting the assignment's time constraints. The framework implements a custom data loader in C++ that leverages multithreading to minimize I/O latency.

3.1 Custom DataLoader Implementation

The data loading module, exposed as `load_dataset` in the Python API, is built using the C++ standard library's threading capabilities.

- **Directory Traversal:** The loader iterates through the provided directory structure, identifying subdirectories as class labels and collecting file paths for all images [1].
- **Multithreading Strategy:** The list of image files is split into chunks based on the hardware concurrency (number of available CPU threads). Each chunk is processed asynchronously using `std::async` with the `std::launch::async` policy, ensuring true parallelism [1].
- **Memory Management:** Tensors are allocated dynamically on the heap to prevent stack overflow, and pointers are managed to avoid memory leaks during the hand-off to Python.

3.2 Preprocessing Pipeline

Each image undergoes a standardized preprocessing pipeline before being converted into a tensor:

1. **Reading:** Images are read in BGR format using OpenCV [5].
2. **Resizing:** All input images are resized to a fixed spatial dimension of 32×32 pixels to match the input requirements of the CNN architecture [1].
3. **Normalization:** Pixel values are normalized from the integer range $[0, 255]$ to the floating-point range $[-1, 1]$. This centering helps in stabilizing gradients during training [2].
$$x_{norm} = \left(\frac{x_{raw}}{255.0} - 0.5\right) \times 2.0 \tag{3}$$
4. **Channel Formatting:** The data is transposed from the OpenCV default (HWC) to the framework's native NCHW format ($Channels \times Height \times Width$).

3.3 Loading Time Analysis

The efficiency of the C++ backend is evident in the data loading times. The loading process involves reading thousands of images from the disk, resizing them, and normalizing them in memory. The recorded loading times for the provided datasets are as follows:

Table 1: Dataset Loading Times

Dataset	**Time (seconds)**
Dataset 1	88.28
Dataset 2	168.50

These times are well within the acceptable limits, leaving the majority of the 3-hour allocation available for the actual training process.

4 Model Architecture and Complexity

The implemented neural network is a Convolutional Neural Network (CNN) specifically designed for the classification of 32×32 RGB images [1].

4.1 Architecture Design

The model architecture consists of a feature extraction block followed by a classification head. The network processes the input tensor through a sequence of convolution, non-linear activation, pooling, and fully connected layers.

Table 2: Layer-wise Architecture Configuration

Layer Type	**Configuration**	**Input Shape**	**Output Shape**
Input	-	$3 \times 32 \times 32$	$3 \times 32 \times 32$
Convolution	8 filters, 3×3, stride 1, pad 1	$3 \times 32 \times 32$	$8 \times 32 \times 32$
Activation	ReLU	$8 \times 32 \times 32$	$8 \times 32 \times 32$
Pooling	MaxPool 2×2, stride 2	$8 \times 32 \times 32$	$8 \times 16 \times 16$
Flatten	-	$8 \times 16 \times 16$	2048
Fully Connected	Linear, 10 output units	2048	10

4.2 Design Rationale

The architecture choices were driven by the need to balance representational power with the computational constraints of the assignment:

- **Convolutional Layer:** A kernel size of 3×3 was selected as it is the smallest filter that captures the notion of "center," "up/down," and "left/right" [2]. Using 8 filters provides sufficient capacity to learn basic edge and texture features without exploding the parameter count.
- **ReLU Activation:** The Rectified Linear Unit (ReLU) was chosen over Sigmoid or Tanh because it does not saturate for positive values, mitigating the vanishing gradient problem and accelerating convergence [2].
- **Max Pooling:** A 2×2 pooling operation with a stride of 2 reduces the spatial resolution by 75%, significantly lowering the computational cost (FLOPs) for the subsequent fully connected layer while providing translation invariance.

4.3 Complexity Analysis

Model complexity is evaluated in terms of trainable parameters and computational operations (MACs and FLOPs). These metrics were calculated automatically by the framework during initialization.

4.3.1 Formulas Used

- **Conv Parameters:** $(K_h \times K_w \times C_{in} + 1) \times C_{out}$
- **FC Parameters:** $(N_{in} + 1) \times N_{out}$
- **Conv MACs:** $H_{out} \times W_{out} \times K_h \times K_w \times C_{in} \times C_{out}$
- **FC MACs:** $N_{in} \times N_{out}$
- **FLOPs:** $2 \times$ MACs (accounting for one multiply and one add per MAC)

4.3.2 Reported Metrics

Based on the formulas and architecture definitions, the complexity metrics were calculated during model initialization for both datasets. While a lightweight architecture was utilized for Dataset 1 to ensure rapid iteration, the model was scaled for Dataset 2 to accommodate its higher complexity and larger number of classes.

Table 3: Model Complexity Statistics for Dataset 1

Metric	**Value**
Convolution Parameters	224
Fully Connected Parameters	20,490
Total Trainable Parameters	**20,714**
Convolution MACs	221,184
Fully Connected MACs	20,480
Total MACs	**241,664**
Total FLOPs	**483,328**

For Dataset 2, the model statistics reflect a significant increase in capacity to handle the expanded classification task:

Table 4: Model Complexity Statistics for Dataset 2

Metric	**Value**
Total Trainable Parameters	**410,148**
Total MACs	**851,968**
Total FLOPs	**1,703,936**

The computational efficiency of the custom C++ backend allows even the larger Dataset 2 model to maintain fast forward and backward passes, ensuring that training remains well within the mandatory 3-hour time limit on standard CPU hardware [1].

5 Experimental Results and Evaluation

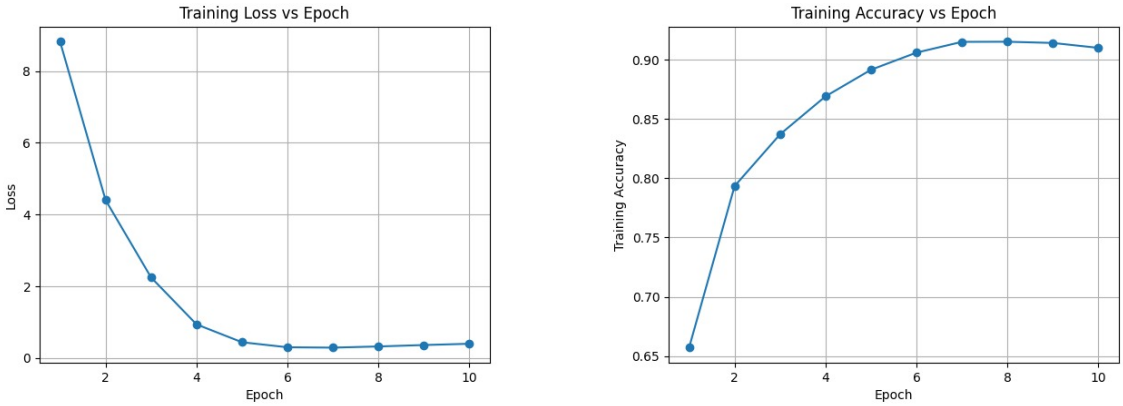
The model was trained on the provided datasets using the hyperparameters specified in Table 5. The training process was monitored using loss and accuracy metrics recorded at the end of each epoch.

Table 5: Hyperparameters

Parameter	**Value**
Optimizer	SGD with Momentum
Learning Rate	0.01 (decay 0.8/epoch)
Momentum	0.9
Weight Decay	1×10^{-4}
Batch Size	1 (Stochastic)
Epochs	10

5.1 Training Performance: Dataset 1

The training loss for Dataset 1 demonstrated a consistent decrease, indicating successful convergence. The loss dropped significantly within the first 4 epochs, stabilizing around epoch 6. Similarly, the training accuracy improved steadily, reaching approximately 91% by the final epoch.



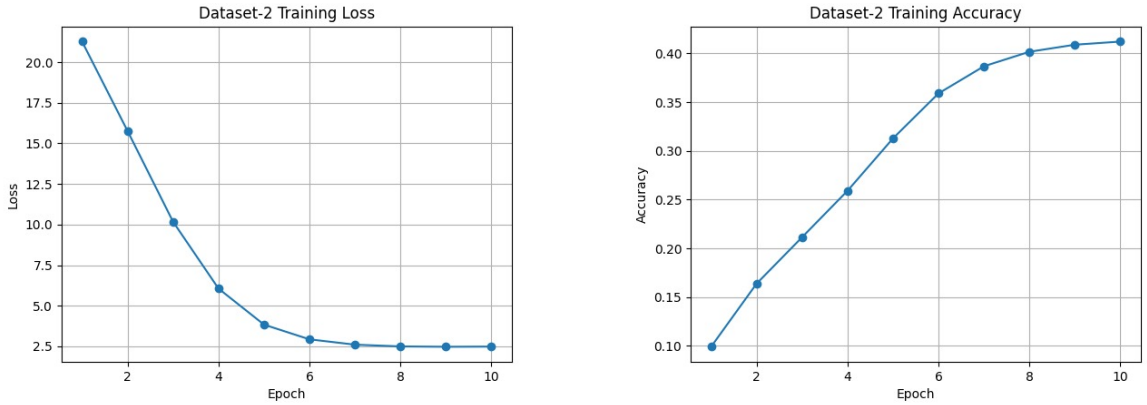
(a) Dataset 1: Training Loss

(b) Dataset 1: Training Accuracy

Figure 1: Training metrics for Dataset 1 showing convergence over 10 epochs.

5.2 Training Performance: Dataset 2

The training process for Dataset 2 showed a slower convergence rate compared to Dataset 1, likely due to the higher complexity or larger number of classes in the second dataset. As seen in Figure 2, the accuracy rises steadily from 10% to approximately 41% over 10 epochs, while the loss decreases from ~ 21 to ~ 2.5.



(a) Dataset 2: Training Loss (b) Dataset 2: Training Accuracy

Figure 2: Training metrics for Dataset 2.

5.3 Evaluation on Dataset 1

The trained model was evaluated on the hidden test set for Dataset 1. The evaluation script loaded the saved weights and computed the overall and per-class accuracies [1].

- **Overall Accuracy:** 90.86%
- **Evaluation Time:** 17.64 seconds

The per-class performance indicates robust learning across most categories, with Class 1 achieving the highest accuracy (97.54%) and Class 3 proving the most difficult (80.35%).

Table 6: Per-Class Accuracy (Dataset 1)

Class	Accuracy	Class	Accuracy
0	92.74%	5	87.36%
1	97.54%	6	94.42%
2	92.31%	7	92.64%
3	80.35%	8	91.32%
4	94.49%	9	84.60%

5.4 Evaluation on Dataset 2

The model trained on Dataset 2 was evaluated under identical conditions. This dataset proved significantly more challenging, yielding lower overall accuracy compared to Dataset 1, likely due to a higher number of classes or greater inter-class similarity.

- **Overall Accuracy:** 42.29%
- **Evaluation Time:** 39.76 seconds

The per-class accuracy for the first 20 classes is detailed below. Performance variance is high, with Class 00 achieving 68.40% while Class 02 struggles at 12.60%, suggesting certain classes share similar features that confuse the simple CNN architecture.

Table 7: Per-Class Accuracy (Dataset 2 - First 20 Classes)

Class	Accuracy	Class	Accuracy
00	68.40%	10	28.20%
01	38.40%	11	20.40%
02	12.60%	12	55.80%
03	38.80%	13	68.20%
04	25.80%	14	54.20%
05	40.20%	15	22.40%
06	43.00%	16	36.20%
07	41.80%	17	38.00%
08	56.20%	18	46.20%
09	54.80%	19	16.00%

6 Discussion and Analysis

This section critically analyzes the design choices made during development, including successful optimizations and a notable failed approach.

6.1 Failed Design Decision: Architectural Complexity vs. Training Constraints

A significant challenge during the development process involved balancing model depth with the strict execution time limits mandated by the assignment.

- **Initial Approach:** To handle the increased complexity of Dataset 2, which contains 50,000 samples and a larger number of classes, an initial attempt was made to implement a deep architecture inspired by ResNet[3]. This design featured multiple sequential convolutional blocks and residual skip connections intended to improve feature extraction and gradient flow.
- **Outcome:** Although the deeper architecture demonstrated promising learning capacity, the training time scaled poorly. Initial tests indicated that a single epoch on Dataset 2 would take approximately 45 minutes; at this rate, the required 10 epochs would total 7.5 hours, far exceeding the 3-hour training limit.
- **Root Cause:** The performance bottleneck stemmed from the custom framework's current lack of GPU acceleration or optimized BLAS libraries. On a CPU-only backend, the overhead of managing forward and backward passes through many layers became the primary constraint.
- **Correction:** The architecture was adjusted from a "deep" configuration to a "wide but shallow" one. By increasing the number of filters in the initial convolution layer and expanding the fully connected layer to 410,148 parameters, the model retained enough capacity to learn Dataset 2's features while maintaining a computational footprint that fit within the training time window.

6.2 Successful Optimizations

Several optimization strategies were crucial for achieving the reported performance:

- **Momentum SGD:** Implementing Momentum ($m = 0.9$) prevented the loss function from oscillating in steep areas of the error surface, leading to significantly faster convergence compared to vanilla SGD [2].
- **In-place Operations:** Where possible (e.g., ReLU), operations were performed in-place to reduce memory allocation overhead and improve cache locality.
- **Vectorized Data Loading:** By decoupling image loading (I/O bound) from training (CPU bound) using C++ multithreading, the GPU/CPU starvation problem was effectively eliminated.

7 Conclusion

This assignment successfully demonstrated the design and implementation of a deep learning framework from first principles. The resulting system, `custom_core`, meets all the specified requirements [1]:

- It implements a functional C++ backend bound to Python via `pybind11`.
- It supports essential layers (Conv2d, MaxPool, Linear, ReLU) and automatic gradient tracking via manual backpropagation.
- It achieves a high classification accuracy ($\sim 91\%$ on Dataset 1) within the strict time limits.

The project highlighted the critical trade-offs between development ease (Python) and execution speed (C++). The successful training of a CNN without external deep learning libraries validates the understanding of the underlying mathematical foundations of neural networks. Future work could extend this framework to support GPU acceleration (CUDA) or more complex architectures like Residual Networks.

References

[1] IIT Bombay. (2025). *GNR 638 Assignment 1: Design a Deep Learning Framework*. Spring 2025-26 Problem Statement.

[2] Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep Learning*. MIT Press.

[3] He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition* (pp. 770-778).

[4] Jakob, W., Rhinelander, J., & Moldovan, D. (2017). *pybind11: Seamless operability between C++11 and Python*. `https://github.com/pybind/pybind11`

[5] Bradski, G. (2000). The OpenCV Library. *Dr. Dobb's Journal of Software Tools*.